

BTU-AI-ASSISTANT -- Proje Raporu

Proje, Bursa Teknik Üniversitesi (BTÜ) öğrenci ve personeline yönelik **yerelde çalışan (on-premise) bir RAG (Retrieval-Augmented Generation) asistanıdır**. Kurum içi veri gizliliğini sağlamak ve gecikme olmadan çalışmak için Ollama üzerinden lokal büyük dil modelleri (LLM) kullanmaktadır.

Kullanılan Teknolojiler ve Kütüphaneler

Sistem, verilerin toplanmasından kullanıcıya sunulmasına kadar modern ve performanslı kütüphanelerle inşa edilmiştir:

- **Büyük Dil Modelleri (LLM):** Yanıt üretimi için yerel olarak çalışan gemma3:12b (Ollama). Veri toplama aşamasında karmaşık PDF tablolarının temizlenmesi için Google Gemini API (gemini-1.5-flash).
- **RAG ve Vektör Veritabanı:** LangChain ekosistemi, ChromaDB vektör veritabanı, HuggingFace Embeddings (paraphrase-multilingual-MiniLM-L12-v2).
- **Gelişmiş Arama (Retrieval):** BM25 (tam kelime eşleşmesi) ve Vektör aramasını birleştiren Ensemble Retriever; ayrıca sonuçları hassaslaştırmak için CrossEncoder Reranker (BAAI/bge-reranker-v2-m3).
- **Web Tarama ve Veri İşleme:** Firecrawl API, requests, BeautifulSoup4, pypdf, pdfplumber.
- **Sunucu ve API:** Flask ve Flask-CORS.

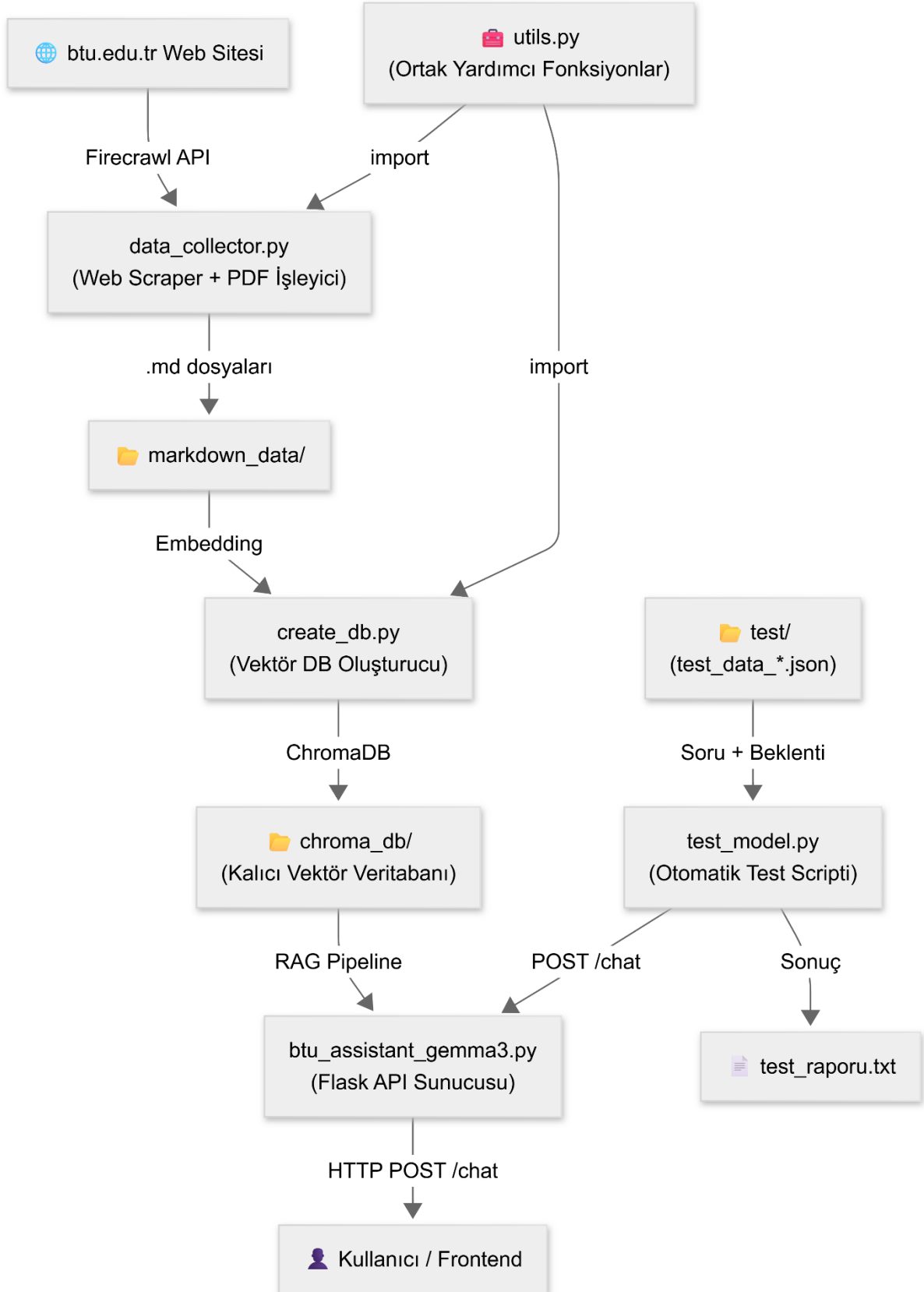
Genel Proje Mimarisi ve Veri Akışı

Aşağıdaki şema, ham verinin web sitesinden toplanıp kullanıcının önüne anlamlı bir cevap olarak gelene kadar izlediği yolu göstermektedir:

Sistemin temel çalışma adımları şunlardır:

1. **Veri Toplama:** btu.edu.tr web sitesi taranarak sayfalar ve PDF belgeleri indirilir.
2. **Veri Temizleme:** Elde edilen karmaşık metinler ve tablolar temizlenerek .md dosyalarına dönüştürülür ve aynı zamanda bir JSON dosyasında (knowledge_base.json) özetlenir.
3. **Vektör Veritabanı:** Temizlenen veriler anlamlandırılır (embedding) ve ChromaDB üzerinde chroma_db/ klasöründe saklanır.
4. **RAG ve API Sunumu:** Kullanıcı soruları alınır, en alakalı belgeler hibrit arama ve reranker algoritmalarıyla filtrelenir, yerel LLM'e bağlam olarak sunulur.

5. **Otomatize Test:** Sistemin doğruluk ve kaynak gösterme oranları düzenli olarak test/ klasöründeki verilerle test edilerek test_raporu.txt içerisine yazılır.



📁 Dizin ve Dosya Yapısı

Projenin istemci (frontend) ve sunucu (backend) bölümlerini barındıran güncel dosya ağacı aşağıdaki gibidir:

```
BTU-AI-ASSISTANT/
├── 🌐 client/
│   ├── 📁 images/
│   ├── 🐘 index.php
│   ├── 📄 script.js
│   └── 🎨 style.css
│       # Kullanıcı Arayüzü (Frontend)
│       # Web sitesi görselleri
│       # Ana sayfa ve tasarım
│       # API istekleri ve dinamik işlemler
│       # Renk, tipografi ve stiller
├── ⚙️ server/
│   ├── 📁 __pycache__/
│   ├── 🗄️ chroma_db/
│   ├── 📁 markdown_data/
│   ├── 📁 reports/
│   ├── 📁 test/
│   │   └── 📄 test_data_all.json
│   ├── 📁 venv/
│   │       # Yapay Zeka ve API (Backend)
│   │       # Derlenmiş Python dosyaları
│   │       # Kalıcı ChromaDB vektör veritabanı
│   │       # Temizlenmiş .md belgeleri
│   │       # Test sonuçlarına ait rapor dosyaları
│   │       # Otomatik performans test veri setleri
│   │       # 50 soruluk test seti
│   │       # İzole Python sanal ortamı
│   ├── 🗝️ .env
│   │       # API anahtarları
│   ├── 🚫 .gitignore
│   │       # Git takibine alınmayacaklar listesi
│   ├── 🐍 btu_assistant_gemma3.py
│   │       # Ana Flask API ve RAG sistemi
│   ├── 🐍 create_db.py
│   │       # Verileri okuyup ChromaDB oluşturan script
│   ├── 🐍 data_collector.py
│   │       # Web scraping ve veri toplama motoru
│   ├── 📄 requirements.txt
│   │       # Gerekli ~150 Python paketinin listesi
│   ├── 🐍 test_model.py
│   │       # Doğruluk ve halüsinasyon ölçüm scripti
│   ├── 📄 test_raporu.txt
│   │       # Sistemin son performans raporu
│   ├── 🐍 utils.py
│   │       # Ortak metin temizleme fonksiyonları
│   └── 📄 visited_urls.json
│       # Ziyaret edilen 1000+ URL'nin geçmişi
├── ⚙️ .gitattributes
│   # Git yapılandırma ayarları
├── ⚖️ LICENSE
│   # Proje lisans bilgileri
└── 📖 README.md
    # Proje kurulum ve kullanım rehberi
```

Detaylı Dosya İncelemesi (Backend)

1. Web Tarayıcısı ve Veri Toplayıcı (data_collector.py)

Amaç:

BTÜ web sitesini (btu.edu.tr) baştan sona tarayıp tüm sayfaları ve PDF belgelerini Markdown formatında markdown_data/ klasörüne kaydeder.

Temel Akış:

Adım	İşlem
1	visited_urls.json'dan geçmiş hafıza yüklenir
2	Firecrawl API ile her URL scrape edilir
3	Linkler çıkarılır, kuyruk dinamik olarak büyütülür
4	İçerik kalitesi denetlenerek metin temizlenir
5	PDF linkleri tespit edilip PDF metni çıkarılır
6	Her belge markdown_data/{başlık}_{hash8}.md olarak kaydedilir

Akıllı Özellikler:

- **İngilizce sayfa filtresi:** /en/ veya /en/ içeren URL'ler otomatik atlanır.
- **Tarih filtresi:** 2025 veya 2026 tarihli olmayan duyuru/haberler should_keep_content() sayesinde atlanır.
- **Hızlı link tarama:** Ziyaret edilmiş URL'lerin içindeki yeni linkleri hafif requests.get() ile bulur (Firecrawl kredisi harcamadan).
- **Kota koruma:** Firecrawl 429/401/quota hatası alındığında döngü durur ve PDF aşamasına geçilir.
- **PDF AI temizleme:** Tablo/ücret/takvim içeren PDF'ler Gemini AI ile temizlenir.

2. Ortak Yardımcı Kütüphane (utils.py)

Amaç: data_collector.py tarafından import edilen yardımcı fonksiyon ve sabitlerin deposudur.

Öne Çıkan Bileşenler:

- **YASAKLI_KELIMELER listesi:** Görsel, arşiv, giriş sayfası, bütçe, ihale, etkinlik gibi 30+ yanlış pozitif tetikleyici içeren URL'leri filtreler.
- **advanced_clean_text(text):** Markdown görselleri, breadcrumb menüleri ve takvim gürültüsünü temizler. seen seti ile tekrar eden satırları kaldırır. Satır atlamalarını korurken fazla boşlukları normalize eder.
- **extract_text_from_pdf(pdf_url):** Önce pdfplumber ile metin çıkarmayı dener, başarısız olursa pypdf (PdfReader) fallback olarak devreye girer. m2/m² sayısını kontrol ederek kroki/plan PDF'leri atar.

- **clean_complex_content_with_llm:** Karmaşık tablo veya ücret listesi içeren metinleri **Google Gemini 1.5 Flash** ile düzenler.
- **should_keep_content(url, content):** Evergreen (ders/bölüm/akademik) sayfaları her zaman tutar, haber sayfalarında güncel yıl kontrolü yapar.

3. Vektör Veritabanı Oluşturucu (create_db.py)

Amaç: markdown_data/ klasöründeki tüm .md dosyalarını okuyup Chroma vektör veritabanına dönüştürür.

Temel Parametreler:

Parametre	Değer
Embedding Modeli	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Chunk Boyutu	800 karakter
Chunk Örtüşmesi	150 karakter
Ayırıcılar	\n\n, # , \n, ., , ""
Chroma DB Yolu	./chroma_db

Çalışma Adımları:

1. **Eski DB temizleme** — shutil.rmtree(CHROMA_PATH)
2. **Embedding modeli yükleme** — HuggingFace multilingual model
3. **Markdown okuma** — Frontmatter (YAML başlık) ayrıştırılır; title, source, type, category çıkarılır
4. **Chunking** — RecursiveCharacterTextSplitter ile parçalama
5. **Bağlam aşılama** — Her chunk'ın başına [KAYNAK BELGE: X | İLGİLİ BİRİM: Y] etiketi eklenir
6. **Chroma'ya kayıt** — Vektörler hesaplanıp diske yazılır

Not: Belgenin URL deseni get_department_from_url ile analiz edilerek /bidb/ (Bilgi İşlem), /sks/ (Sağlık, Kültür ve Spor) gibi ilgili birim dinamik tespit edilir.

4. Ana RAG Asistanı (btu_assistant_gemma3.py)

Amaç: RAG pipeline'ını kurar ve POST /chat uç noktasını sunar.

Sistem Parametreleri:

Bileşen	Değer
Embedding	paraphrase-multilingual-MiniLM-L12-v2
Reranker	BAAI/bge-reranker-v2-m3
BM25 K	12
Vektör K	12
Reranker Top-N	7
Flask Port	5000

Sistem Promptu Kuralları:

1. Sadece sağlanan bağlamı kullan (halüsinasyon yasak)
2. Cevap bulunamazsa "bulamadım" de
3. Listeli sayımlarda cevabı kendin hesapla
4. Her cevabın sonuna kaynak URL ekle
5. Markdown formatında, temiz çıktı ver

API Endpoint: İstemciden (Frontend) gelen istekler JSON formatında POST <http://0.0.0.0:5000/chat> adresine {"message": "Kütüphanede kaç bilgisayar var?"} şeklinde iletilir ve {"status": "success", "reply": "..."} formatında yanıt döner.

5. Otomatik Performans Testi (test_model.py)

Amaç: test/test_data.json içindeki soruları API'ye gönderip 4 metriği ölçer ve rapor üretir.

Ölçülen Metrikler:

Metrik	Hedef
Kaynak Gösterme Oranı	> %90
Doğruluk Oranı	> %85
Halüsinasyon Oranı	< %15
Süre	< 20sn

Akıllı Eşleştirme Algoritması:

metin_normalize_et() fonksiyonu ile Türkçe büyük harf dönüşümü (İ→i, I→ı), noktalama temizliği ve çoklu boşluk sıkıştırma yapılır. Ayrıca muaftır|muafdır|ödemez gibi OR (veya) mantığı ile alternatif kelimeler desteklenerek doğru bilginin haksız yere hata sayılması önlenir.

Çıktı: Test sonuçları test_raporu.txt ve reports/ klasörüne test_raporu_*.txt olarak kaydedilir.

```
=====
Toplam Test Edilen Soru      : 50
Başarılı API Yanıtı         : 50
Ortalama Yanıt Süresi       : 17.58 saniye
Kaynak Gösterme Oranı       : %100.0 (Hedef: > %90)
Doğruluk Oranı              : %88.0 (Hedef: > %85)
Olası Halüsinasyon Oranı    : %12.0 (Hedef: < %15)
=====
```

Tasarım Kararları ve Notlar

Karar	Gerekçe
Hibrit Retrieval (BM25 + Vektör)	Tam kelime eşleşmesi (BM25) ile anlam arama (vektör) birlikte kullanılarak bilgi getirme (recall) oranı artırılmıştır.
CrossEncoder Reranker (BAAI/bge-reranker-v2-m3)	24 aday metinden en iyi 7'sini seçerek, Dil Modeline gönderilen bağlamın kalitesi maksimize edilmiştir.
Lokal LLM (Ollama Gemma3:12b)	Veri gizliliği ve çevrimdışı kullanılabilirlik için Cloud API'ler yerine yerel modeller tercih edilmiştir.
Google Gemini Kullanımı	Karmaşık PDF tablolarını bozmadan düzeltmek için yalnızca data_collector.py (veri hazırlığı) aşamasında kullanılmıştır.